Progetto Heat Dissipation

Paolo Rossi, 235651

2025-06-07

Indice

1 Introduzione **2**

1.1 Fisica del problema 2

2 Algoritmo di calcolo **3**

2.1 Codice parallelo 4

3 Analisi dei dati e conclusioni **5**

3.1 Misure per 1000 step 5

Elenco delle tabelle

1.1 Informazioni tecniche del computer 2

3.1 Tempi di esecuzione in sequenziale 6

3.2 Tempi di esecuzione in parallelo 7

3.3 Tabella dello speed-up 10

Elenco delle figure

1.1 Superficie all'istante iniziale 3

1.2 Scambio termico rispettivamente per 1000 passi e $\varepsilon = 0.0001$ 3

3.1 Grafico Tempo vs Dimensione codice sequenziale 8

3.2 Grafico Tempo vs Dimensione in parallelo 8

3.3 Grafico Thread vs Tempo 9

1 Introduzione

Il presente progetto ha avuto come scopo quello di mettere in evidenza i vantaggi legati al calcolo informatico in parallelo rispetto a quello in sequenziale. In particolare, si richiamerranno i concetti di *speed-up* S (Sezione 3) per dimensionare il confronto tra i due paradigmi computazionali. Inoltre, basandosi sul genere di macchina impiegata, il codice è stato ottimizzato affinché i tempi di esecuzione possano diminuire all'incrementare del grado di paralllelismo, ossia del numero di *thread* impiegati. Il codice è stato sviluppato su diversi dispositivi, tutti con sistema operativo Linux, cionostante le misure raccolte e quindi anche le migliorie apportate sono state svolte su un unico computer, di cui si riportano le informazioni nella tabella sottostante:

Tabella 1.1. Informazioni tecniche del computer

Parametro	Valore
OS	Linux
CPU(s)	32
Thread(s) per core	2
Core(s) per socket	16
Socket(s)	1
Cache L1d	512 KiB (16 instances)
Cache L1i	512 KiB (16 instances)
Cahe L2	16 MiB (16 instances)
Cache L3	64 MiB (2 instances)

1.1 Fisica del problema

Il programma implementato è destinato a calcolare lo scambio del calore di una superficie quadrata. A fini algortimici la superficie viene assunta come una matrice quadrata di dimensione $N \times N$, i cui elementi rappresentano la quantità di calore ceduta ad un certo istante temporale t. In questi termini, il valore di ciascun elemento in riga i-esima e colonna j-esima è regolato dalla seguente equazione differenziale:

$$T_{i,j}^{t+1} = T_{i,j}^{t} + \alpha \cdot \Delta t \cdot \left(T_{i+1,j}^{t} + T_{i,j+1}^{t} + T_{i-1,j}^{t} + T_{i,j-1}^{t} - 4 \cdot T_{i,j}^{t} \right) \tag{1}$$

dove $\alpha = 0.5$ è il coefficiente di scambio termico e $\Delta t = 0.1$ è il passo temporale.

È necessario notare che questa relazione non vale per i bordi della matrice. Dunque si assume che il loro valore sia pari a quello dell'elemento accanto più interno. Da ciò è diretta conseguenza che gli elementi posti ai vertici della matrice non vengano mai attualizzati, per cui rimangono costantemente a valore nullo. Questa implicazione è accettabile, in quanto secondo l'equazione 1 questi ultimi non contribuiscono mai allo scambio termico.

Inoltre, vengo posti a valori costanti gli elementi $T_{5,5} = 5$ e $T_{20,20} = 3$, che possono ritenersi le sorgenti di calore, sicché il calcolo non converga a zero troppo presto.

Come verrà spiegato successivamente, il calcolo può estendersi per un arco temporale predeterminato oppure finché non si raggiunge uno stato sufficientemente stabile degli elementi. La

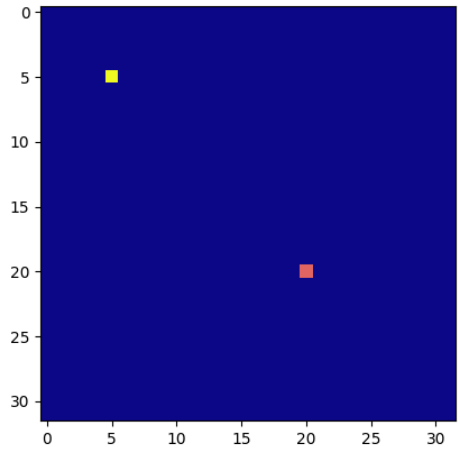

Figura 1.1. Superficie all'istante iniziale

stabilità viene determinata ad ogni istante t come la somma delle differenze tra l'elemento $T^t_{i,j}$ e $T^{t-1}_{i,j}$:

$$\varepsilon = \sum_{i=1}^{N} \sum_{j=1}^{N} \left(T^t_{i,j} - T^{t-1}_{i,j} \right). \tag{2}$$

Quando ε assume un valore sufficientemente piccolo - in base alla dimensione della matrice -, allora termina il processo.

Nelle Figure 1.1 e 1.2 viene visualizzato il fenomeno calcolato.

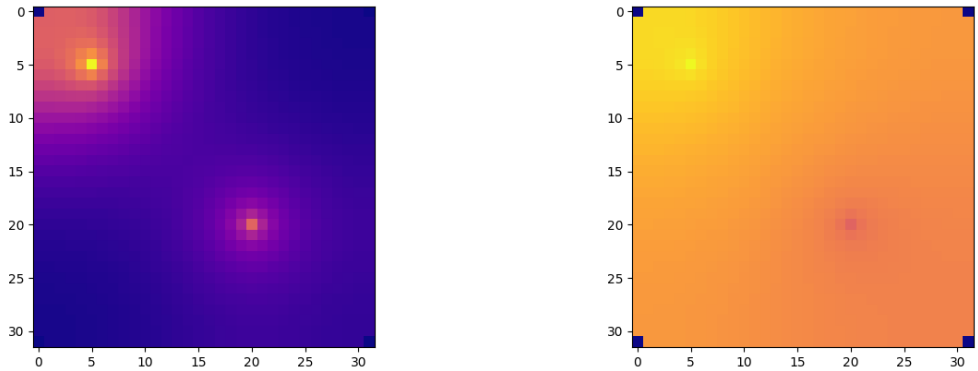

Figura 1.2. Scambio termico rispettivamente per 1000 passi e $\varepsilon = 0.0001$

2 Algoritmo di calcolo

L'implementazione del problema si sviluppa in un ciclo temporale fintantoché non venga raggiunto il numero massimo di passi prestabilito oppure non si converga al valore di ε[1]. Mentre l'attualizzazione dell'intera matrice si articola in cinque stadi:

[1] Sia nel codice sequenziale che in quello parallelo si trova che il ciclo temporale è all'interno di uno più esterno: questo serve a ripetere l'esecuzione dell'algoritmo per la raccolta dati.

1. Copia della matrice in un variabile temporanea.
2. Attualizzazione del corpo della matrice, in cui è implementata l'equazione 1.
3. Ripristino dei valori costanti delle sorgenti di calore.
4. Attualizzazione dei bordi.
5. Calcolo dello scarto ε.
6. Controllo della convergenza dei valori.

È tramite questa routine con cui è stato realizzato il programma sequenziale `seq_main.cc`.

Si aggiunge una piccola nota riguardo a come la matrice viene gestita nel codice: è stato scelto di implementare una classe `Matrix` solamente per semplificare la scrittura e lettura del codice. Per la stessa ragione, inizialmente era stato implementato come attributo un doppio puntatore agli elementi della matrice; successivamente si è passati ad uno singolo, permettendo così un codice più efficiente: infatti in questa configurazione è richiesto un accesso in memoria in meno.

2.1 Codice parallelo

Per quanto concerne il codice in parallelo, `par_main.cc`, la logica di attualizzazione è la stessa e bisogna aggiungere le istruzioni e le direttive di OpenMP per organizzare il parallelismo. Il paradigma seguito è contraddistinto da due peculiarità:

- la prima riguarda il fatto che i *thread* vengono creati e distrutti un'unica volta, prima del ciclo temporale, e quindi vengono riutilizzati ad ogni iterazione di quest'ultimo. Inizialmente si era optati per un'apertura e chiusura dei canali ad ogni step, che naturalmente andava ad appesantire la macchina, la cui diretta conseguenza era un'aumento notevole dei tempi di esecuzione, in alcuni casi addirittura superiori a quelli in sequenziale;
- l'altro aspetto riguarda la distribuzione del lavoro di attualizzazione del corpo della matrice: è stato scelto di suddividere la matrice in blocchi, assegnando ciascuno di essi ad un *thread* specifico in base al loro identificativo `t_ID`. In altre parole, mediante una forte indicizzazione ogni *thread* può operare solo su una determinata porzione di memoria, ovviando così a condizioni di *data race*. Tale configurazione presenta diverse istruzioni per l'identificazione della posizione di ciascun blocco nella matrice globale, però ha il vantaggio di non richiedere nessuna direttiva di sincronizzazione dati, che comporterebbero un aumento dei tempi. Nello specifico la matrice viene suddivisa in un numero di blocchi rettangolari uguali pari a quello dei *thread* disponibili[2], qualora le dimensioni della matrice originale lo permettano. Se, però, la dimensione di ciascun blocco dovesse eccedere quella massima per cui può essere contenuta in cache `L1d`[3], allora la suddivisione avviene per blocchi quadrati di grandezza prefissata - tale per cui sia consentita una certa tolleranza di memoria libera - e si scorreranno i blocchi sulla matrice originale per effettuare l'attualizzazione completa.

[2] La spartizione è stata strutturata considerando la dimensione delle matrici e il numero di *thread* come potenze di 2.

[3] La scelta di considerare la cache più interna è legata al fatto che si volesse limitare il più possibile la lettura e scrittura dei dati sui diversi generi di memoria.

Di seguito vengono riportate le descrizioni riguardanti le scelte di parallelizzazione degli altri punti dell'algoritmo:

1. Per quanto riguarda la copia nella matrice temporanea è stato impiegato un ciclo `for` parallelizzato staticamente e vettorizzato `simd`, in ragion del fatto che l'istruzione è sempre la stessa per ogni iterazione. Viene anche fatto osservare che la copia è stata implementata come metodo della classe `Matrix`, accedendo così direttamente al puntatore che punta all'elemento corrente, senza passare per il sovraccaricamento dell'operatore `[]`, che allungherebbe i tempi.

2. Il ripristino delle sorgenti di calore viene effettuato dal *thread* principale `master`, mentre tutti gli altri vengono messi in attesa. Un'operazione del genere è così breve che non avrebbe motivo di essere parallelizzata.

3. Per l'aggiornamento dei bordi è stato scelto di impiegare una combinazione delle direttive `omp single` e `omp sections`. Dopo una barriera implicita, `single` permette ad un solo *thread* di operare, il quale a sua volta impiega quattro *thread* disponibili per svolgere le istruzioni, mentre quelli rimanenti vengono messi in attesa. Intuitivamente questa configurazione non sembra essere conveniente se vengano impiegati due *thread*, in quanto il numero di sezioni è maggiore dei canali disponibili, cionostante si verifica sperimentalmente che lo *speed up* è comunque maggiore di 1.

4. L'ottimizzazione del calcolo dello scarto è analogo al passo 1., con l'aggiunta di una `reduction` per calcolare il risultato totale di ogni *thread*.

5. Infine vi è la verifica di convergenza, che è gestita solo dal *thread* `master` per evitare che venga svolta inutilmente più volte.

Al termine di ogni passo temporale viene forzata una sincronizzazione, affinché ogni *thread* possa accedere ai dati aggiornati.

3 Analisi dei dati e conclusioni

La raccolta dei tempi di esecuzione sia per il codice sequenziale che per quello parallelo è stato svolto mediando i risultati su 50 esecuzioni[4] per dimensione della matrice e per numero di *thread*. In particolare:

- le dimensioni della matrice sono tutte potenze di 2: $32, 64, 128, 256, 512, 1024$;
- il numero di *thread* è stato scelto in base alla macchina impiegata (Tabella 1.1) e comunque potenza di 2: $2, 4, 8, 16$.

3.1 Misure per 1000 step

Le performance sono state analizzate impostando i passi temporali a 1000. Inoltre sono state scelte due configurazioni parallele che si distinguono per la dimensione massima dei blocchi `MAX_SIZE`, considerando lo spazio disponibile in `L1d` pari a 512 kB (Tabella 1.1):

[4] Dall'analisi statistica sono stati trascurati i primi valori di *warm up*, per i quali il tempo risulta essere superiore poiché era neccessario caricare ancora tutti i dati in cache.

- `MAX_SIZE` = 64: occupa in memoria uno spazio pari a $64 \cdot 64 \cdot$`size_of(double)`$= 64 \cdot 64 \cdot 8$ B = 32.768 kB;
- `MAX_SIZE` = 128: occupa in memoria uno spazio pari a $128 \cdot 128 \cdot <$`size_of(double)`$> = 128 \cdot 128 \cdot 8$ B = 131.072 kB[5].

Di seguito vengono mostrati i risultati delle misure:

Tabella 3.1. Tempi di esecuzione in sequenziale

Size	Mean (s)	SD (s)
32	0.014	1.53e-03
64	0.058	1.86e-04
128	0.233	6.46e-03
256	0.932	8.03e-03
512	3.762	2.24e-02
1024	15.075	9.15e-02

[5] Lo spazio disponibile in cache è ampliamente superiore, ciononostante al fine di mettere in rilievo lo *speed up* sono sufficienti questi valori.

Tabella 3.2. Tempi di esecuzione in parallelo

THD	Size	Mean 64 (s)	SD 64 (s)	Mean 128 (s)	SD 128 (s)
2	32	0.011	1.42e-03	0.011	1.32e-03
2	64	0.039	6.94e-04	0.038	4.21e-03
2	128	0.143	3.54e-04	0.143	3.57e-03
2	256	0.588	7.29e-02	0.780	1.75e-01
2	512	3.227	7.37e-01	3.167	6.72e-01
2	1024	12.472	2.72e+00	10.610	2.46e+00
4	32	0.015	4.27e-04	0.015	1.04e-04
4	64	0.038	2.79e-03	0.036	6.21e-05
4	128	0.130	2.23e-03	0.128	2e-04
4	256	0.507	1.52e-02	0.483	3.23e-04
4	512	2.030	2.6e-02	1.947	1.36e-03
4	1024	7.873	2.34e-01	7.570	1.86e-01
8	32	0.013	3.88e-04	0.012	7.42e-04
8	64	0.027	2.04e-03	0.025	1.06e-03
8	128	0.080	4.63e-03	0.077	4.22e-04
8	256	0.275	1e-02	0.275	3.08e-03
8	512	1.090	3.07e-02	1.032	8.19e-03
8	1024	4.116	9.15e-02	3.920	5.08e-02
16	32	0.014	9.93e-05	0.012	2e-04
16	64	0.023	3.51e-04	0.023	1.86e-04
16	128	0.063	3.51e-04	0.063	2.66e-03
16	256	0.205	1.24e-03	0.205	2.34e-03
16	512	0.762	1.99e-02	0.736	3.38e-03
16	1024	2.989	1.15e-01	2.821	1.21e-01

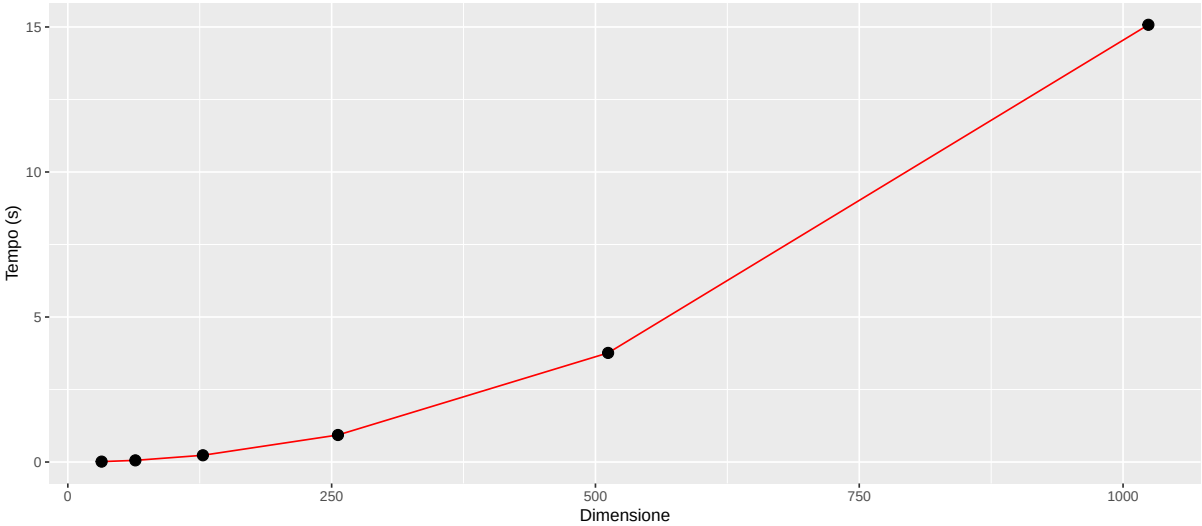

Figura 3.1. Grafico Tempo vs Dimensione codice sequenziale

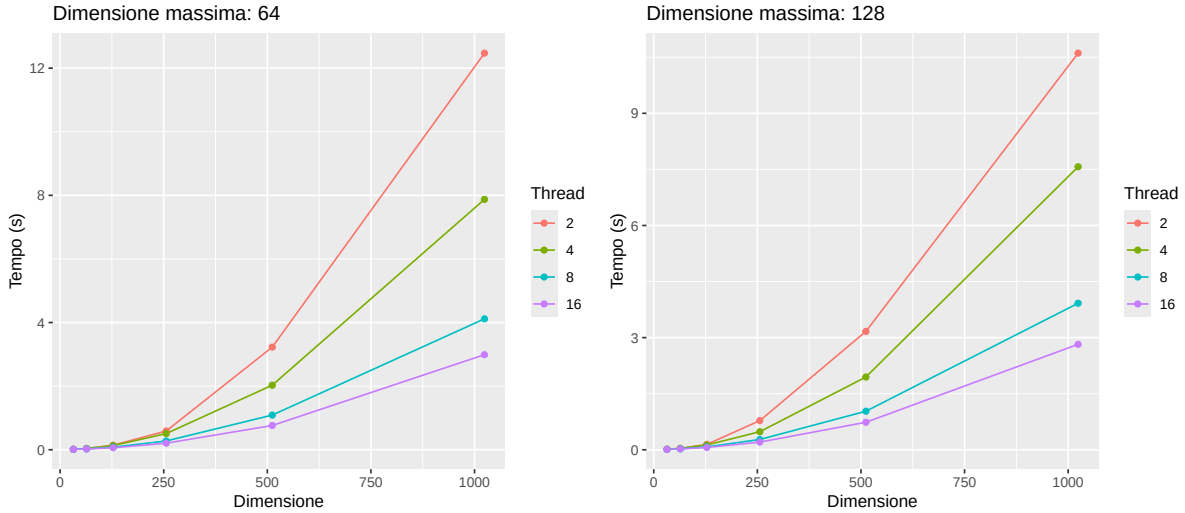

Figura 3.2. Grafico Tempo vs Dimensione in parallelo

Ciò che si evince è che effettivamente i tempi diminuiscono in generale nella configurazione in parallelo rispetto a quella sequenziale (Tabelle 3.1 e 3.2). Peraltro si ha anche una scalatura all'aumentare del grado di parallelismo (Figura 3.3).

Infine è interessante osservare che il tempo scala anche aumentando il valore di `MAX_SIZE` (Figura 3.2): quanto succede è logico in quanto, per il tipo di macchina impiegata, si sfrutta meglio lo spazio disponibile in cache, avendo blocchi più grandi e diminuendo così le iterazioni necessarie per scorrere i blocchi sulla matrice e quindi riducendo le operazioni di lettura e scrittura in memoria. In altri termini avere dei blocchi con più elementi, entro i limiti della cache, permette di trarre vantaggio dal principio di località spaziale dei dati in memoria.

I vantaggi sulle tempistiche possono essere riassunti dal parametro di *speed-up*, definito come il tempo di esecuzione in sequenziale su quello in parallelo:

$$S = \frac{t_{seq}}{t_{par}}. \tag{3}$$

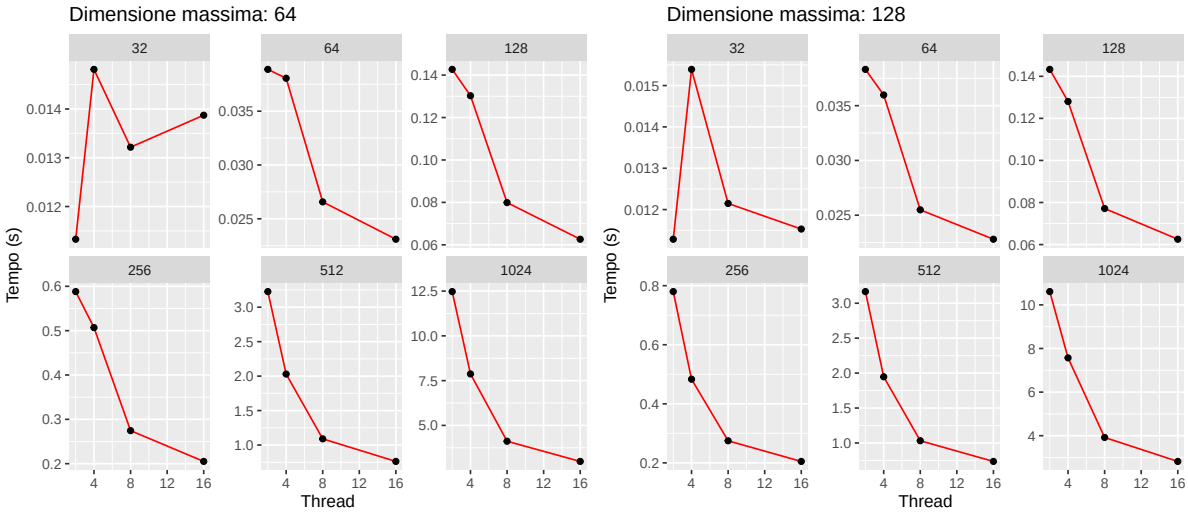

Figura 3.3. Grafico Thread vs Tempo

Infatti in Tabella 3.3 si osserva che:

- il rapporto è in generale maggiore di uno, ad eccezione del caso in cui vengono impiegati quattro *thread*. La ragione potrebbe essere dovuta al passo 4., in cui è presente la ripartizione in sezioni: infatti dei quattro *thread* disponibili, uno è impiegato da `single`, mentre i tre rimanenti devono operare sulle quattro sezioni. A questo punto, avere un numero dispari di *thread* può comportare un *overhead*;
- all'aumentare delle dimensioni della matrice il guadagno in termini di tempo si alza, fino ad arrivare ad un'esecuzione oltre le cinque volte più veloce;
- incrementare il grado di parallelismo permette di avere uno *speed-up* maggiore, specialmente per matrici grandi;
- in generale la configurazione con `MAX_SIZE` maggiore possiede uno *speed-up* leggermente superiore.

Tabella 3.3. Tabella dello speed-up

THD	Size	Speed-up 64	Speed-up 128
2	32	1.29	1.29
4	32	0.95	0.95
8	32	1.09	1.18
16	32	1.02	1.18
2	64	1.48	1.52
4	64	1.52	1.60
8	64	2.14	2.31
16	64	2.51	2.51
2	128	1.63	1.63
4	128	1.79	1.82
8	128	2.91	3.03
16	128	3.70	3.70
2	256	1.59	1.20
4	256	1.84	1.93
8	256	3.39	3.39
16	256	4.55	4.55
2	512	1.17	1.19
4	512	1.85	1.93
8	512	3.45	3.65
16	512	4.94	5.11
2	1024	1.21	1.42
4	1024	1.91	1.99
8	1024	3.66	3.85
16	1024	5.04	5.34